



Barcelona School of Economics

Big Data Management for Data Science

Lab 1: Graph Databases

Aleksandr Smolin
Julian Romero

April 29, 2025

A

A.1. Graph representation

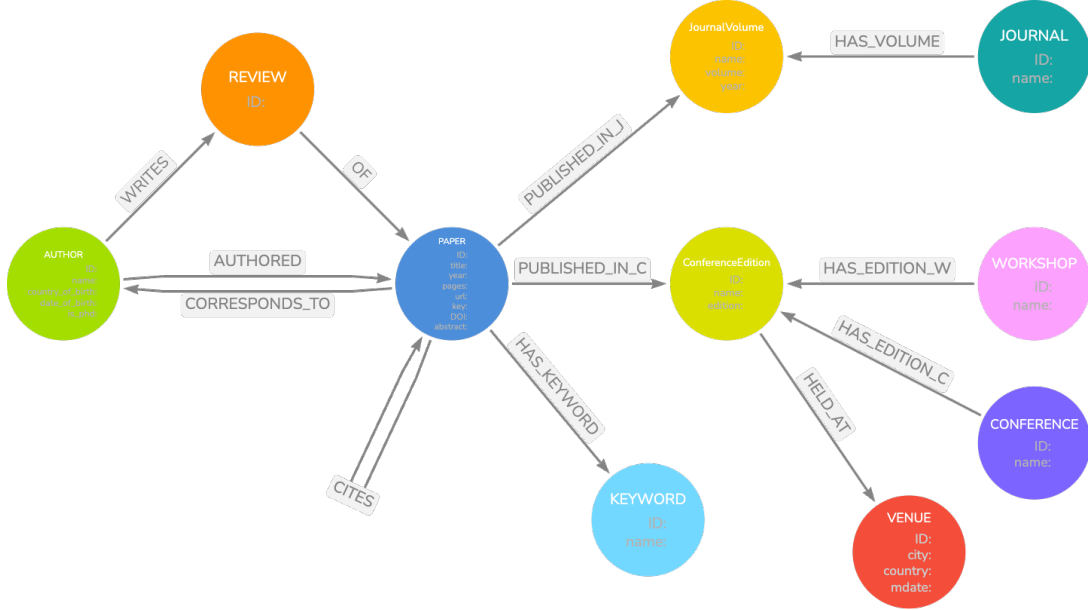


Figure 1: Graph representation with properties

Figure 1 illustrates the graph structure, designed to provide flexibility, extensibility, simplicity, and optimized query performance for Section B. The decision to model **Workshop** and **Conference** as separate node types stems from their distinct nature and the specific requirements of query B1. Although we could have avoided creating separate nodes by simply adding a **type** property to **ConferenceEdition**, this would have required checking the type value for every edition in each query, significantly degrading performance. Moreover, the chosen representation allows for the future addition of attributes specific to workshops and conferences, thereby improving extensibility while maintaining efficient and straightforward queries.

Similarly, **JournalVolume** was modeled separately due to its inherent distinction from **ConferenceEdition** and the specific needs of query B3, which concerns the impact factors of journals. Regarding **Review**, we modeled it as a node rather than a direct relationship, recognizing that a review is an object with its own attributes. Additionally, modifying relationship properties would have been more costly than managing properties within a dedicated node, following A.3. requirements.

The decision to model **Keyword** as a separate node, rather than as a property of **Paper**, also follows this logic: it enables efficient querying of papers associated with the same keywords without having to check each individual paper. The same reasoning applies to **Venue**, where a node was created to facilitate queries across all **ConferenceEdition** instances held at the same location or time. Finally, we included properties for each node that were considered relevant from the data source DBLP (for instance, those associated with **Paper**), synthesized new attributes where necessary (such as those for **Author**), and left some nodes initialized with only an ID, allowing for future extensions.

A.2. Data loading

After downloading and transforming the raw DBLP data from `.xml` to `.csv` format, we preprocessed and adapted it to meet the project requirements using Python with the Polars library. This approach enabled us to efficiently manage millions of records, create new properties, and generate additional nodes and relationships, which were subsequently exported into `.csv` files. To load the data into Neo4j, we employed a bulk import using `neo4j-admin database import`, as Cypher's `LOAD CSV` expressions are not designed to efficiently handle datasets of this scale. The entire process is thoroughly documented in the `main.ipynb` notebook, including detailed comments and instructions to facilitate effective population of the database.

Regarding the assumptions made for the synthetic data, citations were generated following a real-world distribution, specifically a power-law distribution, for 0.5% of the final papers. This proportion was deemed sufficient for the purposes of the exercise, as generating citations for all papers would have only added unnecessary computational burden. For keywords, a list of 20 distinct terms was manually created and randomly assigned to each paper, with each paper receiving between 1 and 3 keywords. Finally, to generate reviews, the top 50 authors were first selected based on their number of publications. Reviews were then randomly assigned, with each paper receiving between 1 and 5 reviews, ensuring that none of the reviewers were also authors of the corresponding paper. This approach follows the exercise instructions and maintains consistency throughout the dataset.

A.1. Graph extension

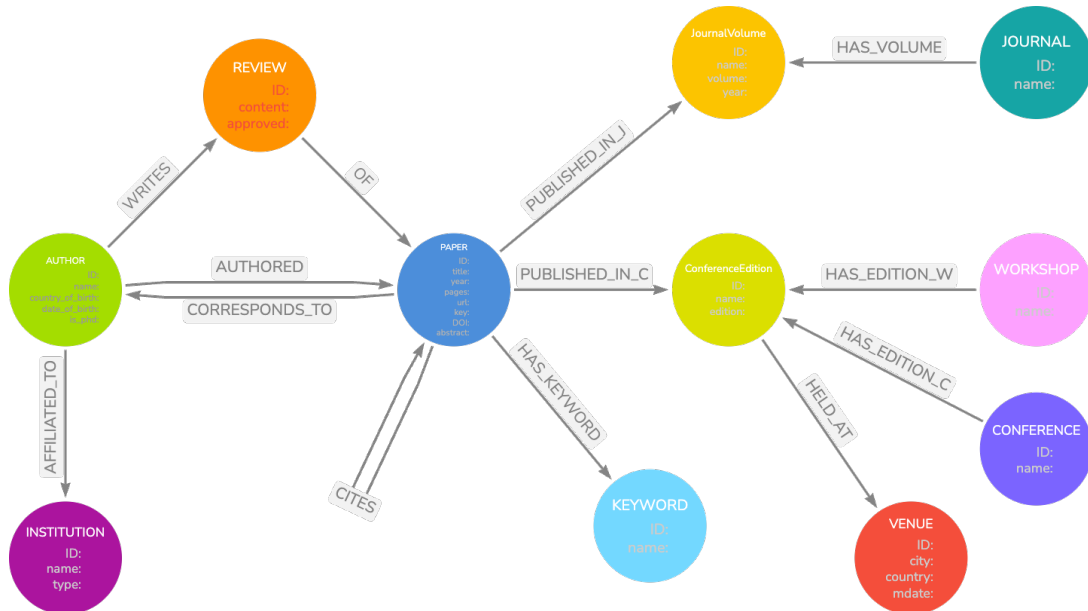


Figure 2: Extended graph model

Figure 2 displays the extended graph model, including content and approved properties in `Review` node, and also the `Institution` node and `AFFILIATED_TO` edge. The reasons behind this was to facilitate the search of authors affiliated to the same institutions, and

the extensibility of the properties of the review node as an object itself.

To implement this, fake data was created. To be consistent, all reviews have approved in True given we are dealing with all published papers. On the other hand, `Institution` we manually generated with a type property distinguishing between universities and companies. Finally, data was imported again to the database using the bulk import tool using `neo4j-admin` command, clearly detailed and described in `main.ipynb` notebook.

B. Querying the data

In the following subsections the queries can be found. They are also displayed with their corresponding output in `main.ipynb`.

B.1. Top 3 most cited papers of each conference

```
// Match papers published in conferences
MATCH (conf:Conference)-[:HAS_EDITION_C]->(ed:ConferenceEdition),
(ed)<-[:PUBLISHED_IN_C]-(p:Paper)
// Match papers that are cited
MATCH (p)<-[:CITES]-(c)
// Count citations grouping by paper and conference
WITH conf.name AS confName, p, count(c) AS citationCount
// Order by conference and citation descending, keeping most cited papers first
ORDER BY confName, citationCount DESC
// Keep only the top 3 most cited papers per conference
WITH confName, collect({paper: p, citations: citationCount})[0..3] AS top3
// Ungroup the top 3 papers to return them in separate rows
UNWIND top3 AS entry
// Return the conference ID, paper title, and citation count
RETURN
    confName          AS Conference,
    entry.paper.title  AS Paper,
    entry.citations    AS CitationCount
// Order by conference and citation count descending
ORDER BY CitationCount DESC, Conference;
```

B.2. Community per conference

```
// find those authors that published in a conference edition of the type conference
MATCH (conf:Conference)-[:HAS_EDITION_C]->(ed:ConferenceEdition),
(ed)<-[:PUBLISHED_IN_C]-(p:Paper)<-[:AUTHORED]-(a:Author)
// Group by conference name and author name,
// and count the number of editions the author appeared in the same conference
WITH
    conf.name          AS conference,
    a.name              AS author,
    COUNT(DISTINCT ed.name) AS editionsCount
// Keep only those authors that published in at least 4 editions of the same conf
```

```
WHERE editionsCount >= 4
RETURN conference, author, editionsCount
ORDER BY conference, editionsCount DESC;
```

B.3. Impact factors

```
// Gather, per journal, all papers published in 2022 or 2023
MATCH (j:Journal)-[:HAS_VOLUME]->(vol:JournalVolume)-[:PUBLISHED_IN_J]-(p:Paper)
WHERE vol.year IN [2022, 2023]
WITH j, collect(p) AS prevPapers, size(collect(p)) AS numPrevPapers

// Count incoming citations in 2024 to any of those papers
UNWIND prevPapers AS oldPaper
MATCH (citing:Paper)-[:CITES]->(oldPaper)
WHERE citing.year = 2024
WITH j, numPrevPapers, count(citing) AS numCitations

// Compute impact factor, guard against division by zero
RETURN
j.name          AS Journal,
numCitations     AS Citations2024,
numPrevPapers    AS Papers2022_23,
CASE
  WHEN numPrevPapers > 0
  THEN toFloat(numCitations) / numPrevPapers
  ELSE NULL
END
ORDER BY ImpactFactor2024 DESC;
```

B.4.H-indexes

```
// Calculate per-paper citation counts
MATCH (a:Author)-[:AUTHORED]->(p:Paper)
OPTIONAL MATCH (p)-[:CITES]-()
WITH a, p, count(*) AS citations

// Get all that author's citation counts into a list
WITH a, collect(citations) AS citationList

// Order the list in ascending order by using a custom method
UNWIND citationList AS citationCount
WITH a, citationCount
ORDER BY citationCount ASC

// Reverse the list (ascending order to descending order)
WITH a, collect(citationCount) AS sortedList

// Unwind positions and keep only those where citationCount >= rank
```

```
UNWIND range(0, size(sortedList)-1) AS idx
WITH a, idx + 1 AS h, sortedList[idx] AS citationCount
WHERE citationCount >= h

// The h-index is the maximum valid h, defaulting to 0 if none
RETURN a.name AS Author, coalesce(max(h), 0) AS HIndex
ORDER BY HIndex DESC, Author;
```

C. Data Science Algorithm

```
CALL gds.graph.project(
  'citation-net',
  ['Paper'],
  {
    CITES: {
      type: 'CITES',
      orientation: 'NATURAL'
    }
  }
);
CALL gds.pageRank.stream(
  'citation-net',
  {
    maxIterations: 50,
    dampingFactor: 0.85
  }
)
YIELD nodeId, score

WITH gds.util.asNode(nodeId) AS paper, score

RETURN
paper.title AS title,
paper.year AS year,
paper.pages AS pages,
paper.url AS url,
paper.DOI AS DOI,
score AS score
ORDER BY score DESC
LIMIT 8;
```

We have applied the PageRank algorithm to our citation network to identify influential papers based on their citation structure. The Cypher query above projects a graph based on the Paper nodes and CITES relationships, and then runs the PageRank algorithm to compute scores for each paper. The papers are ranked by their importance based on the citation network.

The PageRank algorithm calculates a score for each paper based on its influence within the citation network, with higher scores indicating greater importance. The query returns the top 8 papers, showing their titles, publication years, page ranges, URLs, DOIs, and scores. It's important to note that the citation data is artificial, so the values do not reflect real-world citations. Despite identical PageRank scores (0.405), these papers are considered equally influential, highlighting a well-connected set of works across various academic domains, from 1999 to 2022. From a domain perspective, these results suggest that both older and more recent papers are highly relevant, indicating a continuous flow of impactful research that shapes the field, regardless of publication year.